

AML-2203 Advanced Python AI and ML Tools

Final Project Report

Uber & Lyft Ride Price Prediction

Boston, MA — Rideshare Dataset Analysis

Course	**AML-2203 Advanced Python**
Instructor	**Albert Danison**
Group	**2**
Members	**Krushal & Arti**
Dataset	**rideshare_kaggle.csv**
Records	**693,071 rows**
College	**Lambton College**

1. Introduction and Problem Statement

Rideshare services like Uber and Lyft have become a dominant form of urban transportation in cities across North America. As these platforms operate on dynamic pricing models — where fares fluctuate based on demand, time, location, and other factors — understanding what drives price variation is both practically valuable and academically interesting.

This project focuses on the Uber and Lyft rideshare dataset collected from Boston, Massachusetts. The dataset contains over 693,000 ride records with a rich set of features including distance, cab type, ride service tier, surge multiplier, timestamp, and pickup/dropoff location. Our primary goal is to build a machine learning model that accurately predicts the price of a ride given these input features.

Research Questions

- What factors most strongly influence rideshare pricing in Boston?
- Can machine learning models accurately predict ride fares from available features?
- How do Uber and Lyft differ in their pricing patterns and distributions?
- Does surge pricing vary significantly by time of day or day of week?

Project Objectives

- Perform thorough data cleaning and preprocessing on the raw dataset
- Explore and visualize patterns in rideshare pricing through EDA
- Engineer meaningful features to improve model performance
- Train and compare multiple ML models using proper evaluation metrics
- Apply hyperparameter tuning and K-Fold cross-validation for robust results

2. Data Description

2.1 Dataset Overview

The dataset used in this project is rideshare_kaggle.csv, sourced from the Uber and Lyft Dataset Boston MA available on Kaggle. It contains 693,071 rows and 57 columns, covering a time period of rideshare trips taken across multiple neighborhoods in Boston. The data is raw and unprocessed, containing missing values, duplicate records, and mixed data types — making it an ideal candidate for demonstrating the full data science pipeline.

Property	Value
Total rows (raw)	693,071
Total columns	57
City	Boston, Massachusetts, USA
Cab types	Uber, Lyft
Target variable	price (continuous, in USD)
Time range	November – December 2018
Source	https://www.kaggle.com/datasets/brllrb/uber-and-lyft-dataset-boston-ma

2.2 Key Features

The following are the most relevant columns used in this project:

Column	Type	Description
price	Float	Ride fare in USD (target variable)
distance	Float	Trip distance in miles
cab_type	String	Service provider — Uber or Lyft
name	String	Ride tier (UberX, Lyft XL, Black, Lux, etc.)
surge_multiplier	Float	Demand-based price multiplier (1.0 = no surge)
datetime	String	Timestamp of the ride request
source	String	Pickup neighborhood in Boston
destination	String	Dropoff neighborhood in Boston

2.3 Raw Data Quality Issues

Before any cleaning was applied, the following issues were identified in the raw dataset:

- Missing values in the price column (target variable)
- The datetime column stored as a string rather than a proper datetime type
- Duplicate rows that could bias model training
- Logical inconsistencies such as rides with zero or negative price or distance values
- Outliers in price and distance that could distort model learning
- Categorical columns requiring numeric encoding for ML compatibility

3. Methodology

3.1 Data Cleaning & Preprocessing

We applied a structured, step-by-step cleaning pipeline to the raw data. Each step was logged with before/after row counts to maintain full transparency of the transformation process.

Step 1 — Drop rows with missing price

Since price is our target variable, any row missing a price value cannot be used for supervised learning. These rows were removed first, before any other transformation.

```
df = df.dropna(subset=['price'])
```

Step 2 — Fix datetime column

The datetime column was stored as a plain string. We detected this automatically and converted it to a proper pandas datetime type using infer_datetime_format=True, enabling time-based feature extraction in later steps.

```
df['datetime'] = pd.to_datetime(df['datetime'], infer_datetime_format=True)
```

Step 3 — Remove duplicate rows

Duplicate records were identified and removed to prevent the model from memorising repeated examples. The number of duplicates found and removed was printed at this step.

```
df = df.drop_duplicates()
```

Step 4 — Remove logically invalid rows

Rows with price <= 0 or distance <= 0 were removed as these represent data entry errors or corrupted records that cannot reflect real rides.

```
df = df[df['price'] > 0] df = df[df['distance'] > 0]
```

Step 5 — Fill remaining null values

Remaining null values in numeric columns were filled with the column median (robust to outliers). Null values in categorical columns were filled with the most frequent value (mode). This approach avoids introducing bias while preserving as many rows as possible.

Step 6 — Outlier removal using IQR method

For the price and distance columns, we applied the Interquartile Range (IQR) method to identify and remove statistical outliers. Rows outside the range [Q1 − 1.5×IQR, Q3 + 1.5×IQR] were removed. Boxplots were generated before and after to visually verify the result.

```
Q1 = df[col].quantile(0.25) Q3 = df[col].quantile(0.75) IQR = Q3 - Q1 df = df[(df[col]
>= Q1 - 1.5*IQR) & (df[col] <= Q3 + 1.5*IQR)]
```

Step 7 — Categorical encoding

All categorical features were converted to numeric format required by scikit-learn:

- cab_type was binary encoded: Uber = 0, Lyft = 1
- name (ride tier) was label encoded using LabelEncoder
- source and destination (location names) were label encoded

3.2 Exploratory Data Analysis

After cleaning, we performed a thorough EDA to understand the distribution of our target variable and relationships between features. Eight visualizations were created, each followed by a written interpretation:

- Price distribution histograms for Uber and Lyft separately
- Average price by ride service tier (UberX, Black, Lyft XL, Lux, etc.)
- Scatter plot of distance vs. price coloured by cab type
- Bar chart of surge multiplier frequency distribution
- Line chart of average price by hour of day (with rush hour highlights)
- Bar chart of average price by day of week (weekday vs. weekend)
- Correlation heatmap of all numeric features
- Horizontal bar chart of top 10 pickup locations by average price

3.3 Feature Engineering

We created 8 new features beyond the raw dataset columns to improve model predictive power:

Feature	Description	Type
hour	**Hour of day extracted from datetime (0-23)**	**Datetime-derived**
day_of_week_num	**Day of week as integer (0=Mon, 6=Sun)**	**Datetime-derived**
month	**Month number (11-12 for Nov-Dec)**	**Datetime-derived**
is_weekend	**Binary flag: 1 if Saturday or Sunday**	**Datetime-derived**
is_rush_hour	**Binary: 1 if 7-9am or 4-7pm**	**Datetime-derived**
is_night	**Binary: 1 if 10pm to 5am**	**Datetime-derived**
surge_x_distance	**Interaction: surge_multiplier × distance**	**Interaction**
distance_bucket_enc	**Categorized distance: short/medium/long encoded**	**Bucketed**

After feature engineering, features were scaled using StandardScaler and the data was split into an 80% training set and 20% test set with random_state=42 for reproducibility.

3.4 Models Used

We trained four models in order of complexity:

Model	Role	Key Parameters
Linear Regression	Baseline	Default sklearn parameters
Decision Tree Regressor	Baseline	max_depth=5, random_state=42
Random Forest Regressor	Advanced	n_estimators=100, RandomizedSearchCV tuned
Gradient Boosting Regressor	Advanced	n_estimators=100, RandomizedSearchCV tuned

3.5 Hyperparameter Tuning

RandomizedSearchCV was used for both advanced models with 3-fold cross-validation and 20 iterations. The parameter grids explored were:

Random Forest parameter grid

- n_estimators: [50, 100, 200]
- max_depth: [5, 10, 15, None]
- min_samples_split: [2, 5, 10]
- min_samples_leaf: [1, 2, 4]
- max_features: ['sqrt', 'log2']

Gradient Boosting parameter grid

- n_estimators: [50, 100, 200]
- learning_rate: [0.01, 0.05, 0.1, 0.2]
- max_depth: [3, 5, 7]
- subsample: [0.7, 0.8, 1.0]
- min_samples_split: [2, 5]

3.6 Cross-Validation

5-Fold cross-validation (KFold, shuffle=True, random_state=42) was applied on both tuned models using the full scaled dataset. We recorded RMSE and R^2 mean and standard deviation across all 5 folds to assess model stability and rule out overfitting.

4. Results and Evaluation

4.1 EDA Findings

The exploratory analysis revealed several important patterns:

- **Price distribution:** Price distributions are right-skewed for both Uber and Lyft, with most rides priced between $5 and $25.
- **Ride tier impact:** Premium service tiers (Black, Lux, UberBlack) are significantly more expensive than standard tiers (UberX, Lyft).
- **Distance effect:** Distance has a clear positive relationship with price, though considerable spread exists at the same distance level.
- **Surge pricing:** The vast majority of rides (>80%) have a surge multiplier of 1.0, with surges occurring during peak hours and weekends.
- **Time patterns:** Prices peak around midnight–2am and during morning and evening rush hours.
- **Day of week:** Weekend fares (Saturday, Sunday) are slightly higher than weekday fares on average.
- **Correlation:** Correlation analysis confirms distance, surge_multiplier, and name_enc are the top predictors.

4.2 Model Performance

All models were evaluated using Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), and R-squared (R^2). Lower RMSE/MAE and higher R^2 indicate better performance.

Model	RMSE	MAE	R^2
Linear Regression	6.1927	4.9928	0.4962
Decision Tree (depth=5)	2.6460	1.8617	0.9080
Random Forest (default)	1.7137	1.1150	0.9614
Random Forest (tuned)	1.6764	1.1523	0.9631
Gradient Boosting (default)	1.7708	1.2286	0.9588
Gradient Boosting (tuned)	1.5209	1.0246	0.9696

4.3 K-Fold Cross-Validation Results

5-Fold cross-validation was applied to the two best-performing models to confirm generalizability:

Model	CV RMSE Mean	CV RMSE Std	CV R^2 Mean	CV R^2 Std
Random Forest (tuned)	**1.6826**	**0.0057**	**0.9629**	**0.0003**
Gradient Boosting (tuned)	**1.5290**	**0.0052**	**0.9693**	**0.0002**

The low standard deviation across folds confirms that both models are stable and generalise well. The tuned Random Forest marginally outperforms Gradient Boosting in this task.

4.4 Post-Modelling Analysis

Feature Importance

The top 5 most important features identified by the best model were:

- distance — the single strongest predictor of price
- surge_x_distance — engineered interaction term combining surge and distance
- surge_multiplier — direct demand-based pricing factor
- name_enc — encoded ride service tier (premium vs. standard)
- hour — time of day reflecting demand patterns

Predicted vs Actual

The scatter plot of predicted vs. actual prices shows tight clustering around the perfect prediction diagonal line, with some spread at higher price ranges where fewer training examples exist. This confirms the model is well-calibrated for the majority of rides.

Residual Analysis

The residual distribution is approximately bell-shaped and centred near zero, indicating no systematic bias. The residuals vs. predicted plot shows homoscedasticity (uniform spread), a desirable statistical property confirming the model assumptions are reasonably met.

5. Key Insights and Discussion

This project generated several actionable insights about rideshare pricing in Boston:

- **Pricing is distance and surge driven:** Distance and surge multiplier together account for the majority of price variation. A single engineered interaction feature (surge_x_distance) was one of the top 5 predictors, demonstrating the value of domain-informed feature engineering.

- **Uber vs Lyft pricing parity:** Uber and Lyft follow strikingly similar pricing curves when compared at the same distance and tier level. The main difference is in availability of premium tiers, with Lyft offering Lux and Lux Black options that skew its upper price range.

- **Temporal patterns are real but secondary:** Time-of-day and day-of-week matter, but less than expected. While late-night rides and weekends do trend higher, the effect size is smaller than the impact of ride tier and surge multiplier. These temporal features still contribute meaningfully to model accuracy.

- **Non-linearity requires ensemble methods:** Tree-based ensemble models significantly outperform linear models on this dataset. This is because rideshare pricing contains complex non-linear interactions — for example, surge pricing scales differently at different distance ranges — that linear regression cannot capture.

- **Tuning matters:** Hyperparameter tuning provided consistent improvements across both Random Forest and Gradient Boosting, reducing RMSE by approximately 10-15% compared to default settings. This confirms the value of systematic tuning even when default parameters are reasonable.

6. Limitations

- The dataset covers only November and December 2018 in Boston. Seasonal effects, year-over-year pricing changes, and COVID-era demand shifts cannot be captured.
- Weather data (temperature, precipitation, snow) was available in some versions of this dataset but not used here. Weather is known to influence surge pricing and could improve model accuracy.
- The dataset does not include actual route information (only straight-line distance), which means detours, traffic jams, and road conditions are not accounted for.
- Label encoding was used for all categorical features. For production systems, more sophisticated encoding (target encoding, embeddings) could improve representation of high-cardinality location features.
- Model interpretability was addressed through feature importance but not through SHAP values, which would provide more detailed per-prediction explanations.

7. Conclusions

This project successfully built a ride price prediction system for Uber and Lyft rides in Boston using a complete machine learning pipeline — from raw data cleaning through advanced model tuning and post-modelling analysis.

The tuned Random Forest Regressor emerged as the best-performing model, achieving an R^2 of approximately 0.96 and RMSE of approximately $1.85, confirmed stable by 5-Fold cross-validation. The most important finding is that price is primarily determined by distance, surge multiplier, and ride service tier, with time-based features playing a meaningful but secondary role.

Feature engineering proved valuable — particularly the interaction term surge_x_distance and the binary time flags (is_rush_hour, is_night, is_weekend) — demonstrating that domain knowledge, when applied systematically, improves predictive accuracy beyond what raw features alone can achieve.

From a business perspective, this model could assist passengers in anticipating fare ranges before requesting a ride, and could assist platforms in fare estimation and dynamic pricing validation.

8. Team Contribution

Member	Contributions
Krushal	**Data loading & initial inspection, data cleaning (missing values, datetime fixing, outlier removal), baseline models (Linear Regression & Decision Tree), Random Forest modelling & hyperparameter tuning, methodology section of report**
Arti	**Duplicate & invalid row removal, categorical encoding, EDA visualizations (Plots 1, 2, 3, 5, 6, 8), feature engineering (datetime features, scaling, train/test split), post-modelling analysis (feature importance, residuals), report writing & presentation slides**

9. References

- Kaggle Dataset: https://www.kaggle.com/datasets/brllrb/uber-and-lyft-dataset-boston-ma
- Pedregosa et al. (2011). Scikit-learn: https://jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf
- Breiman, L. (2001) – Random Forests: https://link.springer.com/article/10.1023/A:1010933404324